

Redux Toolkit Query with React & TypeScript

A practical reference for managing server state, fetching posts, and executing mutations using RTK Query.

WHAT RTK QUERY SOLVES

RTK Query is the data-fetching and caching toolset within Redux Toolkit, purpose-built for **server state**—data hosted on an API that must be fetched, cached, synchronized, and presented in UI components.

Without RTK Query, applications must manually orchestrate standard request cycles:

- Tracking loading, success, and error states manually
- Handling duplicate network requests and cache lifetime
- Refetching stale data after write/mutation operations

Client State vs. Server State

State Type	Examples	Typical Tool
Client State	Counters, modal visibility, selected tab	Redux slice, React <code>useState</code> , Context
Server State	Posts returned by an HTTP API, user profiles	RTK Query

FEATURES DEMONSTRATED

- TypeScript models for API requests/responses
- `createApi` and `fetchBaseQuery` setup
- Typed Query endpoints (`GET /posts`)
- Typed Mutation endpoints (`POST /posts`)

- Auto-generated custom React hooks
- Component loading and error handling
- Store reducer and middleware integration
- Redux DevTools integration out of the box

INSTALLATION & PROJECT LAYOUT

Commands:

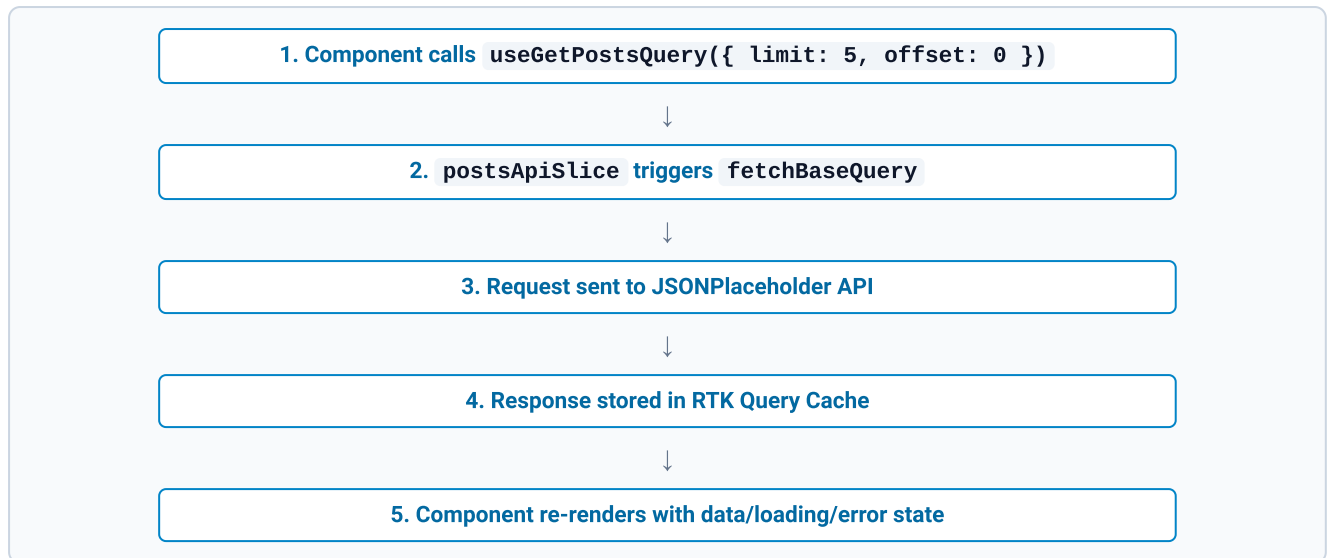
```
npm install
npm run dev

# Utility Scripts
npm run lint
npm run build
npm run preview
```

Project Structure:

```
src/
  components/
    PostsList.tsx
  state/
    counter/CounterSlice.ts
    posts/postsApiSlice.ts
  store.ts
  types/Post.ts
  App.tsx & main.tsx
```

THE REQUEST FLOW



1. DEFINING THE API SLICE

The API slice in `src/state/posts/postsApiSlice.ts` centralizes endpoints and base configurations:

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
import type { Post } from "../../types/Post";

export const postsApiSlice = createApi({
  reducerPath: "posts",
  baseQuery: fetchBaseQuery({
    baseUrl: "https://jsonplaceholder.typicode.com",
  }),
  endpoints: (builder) => ({
    getPosts: builder.query<Post[], { limit: number; offset: number }>({
      query: ({ limit, offset }) => `/posts?_limit=${limit}&_offset=${offset}`,
    }),
    createPost: builder.mutation<Post, Omit<Post, "id">>({
      query: (post) => ({
        url: "/posts",
        method: "POST",
        body: post,
      }),
    }),
  }),
});

export const { useGetPostsQuery, useCreatePostMutation } = postsApiSlice;
```

Core Concepts

- **createApi** : Core entry point defining endpoints, cache keys, and fetch routines.
- **reducerPath** : Unique key in the Redux store state (e.g., `state.posts`).
- **baseQuery** : Lightweight wrapper around standard `fetch` ; prepends base URL.
- **Queries vs Mutations**: Queries represent read operations, while Mutations handle data writes (`POST` , `PUT` , `DELETE`).

2. REGISTERING RTK QUERY IN THE STORE

Both the reducer and custom middleware must be registered inside `src/state/store.ts`:

```
export const store = configureStore({
  reducer: {
    counter: counterReducer,
    [postsApiSlice.reducerPath]: postsApiSlice.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(postsApiSlice.middleware),
});
```

Reducer Responsibility

Stores cached query responses, request statuses, error objects, active subscriptions, and metadata.

Middleware Responsibility

Manages cache lifetimes, handles subscription counts, triggers polling, and executes network requests.

3. PROVIDING THE STORE TO REACT

Wrap the root component tree in `src/main.tsx` with Redux's `Provider`:

```
createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>,
);
```

4. READING DATA WITH QUERY HOOKS

In `src/components/PostsList.tsx`, consume the generated hook to access state automatically:

```
const { data: posts, isLoading, isError } = useGetPostsQuery({
  limit: 5,
  offset: 0,
});
```

Query Status Properties

Field	Meaning
<code>data</code>	The most recently fulfilled response payload
<code>isLoading</code>	First request is actively pending (no cache available)
<code>isFetching</code>	Request is pending (includes background updates)
<code>isSuccess</code> / <code>isError</code>	Flags indicating request outcome

5. WRITING DATA WITH MUTATION HOOKS

Mutations return a trigger function and state object. Use `.unwrap()` for standard Promise behavior:

```
const [createPost, { isLoading: isCreatingPost }] = useCreatePostMutation();

// Standard handle with try/catch
try {
  const createdPost = await createPost({ title: "My Post Created!" }).unwrap();
  console.log("Success:", createdPost);
} catch (error) {
  console.error("Unable to create post:", error);
}
```

AUTOMATED CACHING & TAG INVALIDATION

Enhance the API slice with tags to automatically trigger refetches after mutations occur:

```
export const postsApiSlice = createApi({
  reducerPath: "posts",
  baseQuery: fetchBaseQuery({ baseUrl: "https://jsonplaceholder.typicode.com" }),
  tagTypes: ["Post"],
  endpoints: (builder) => ({
    getPosts: builder.query<Post[], { limit: number; offset: number }>({
      query: ({ limit, offset }) => `/posts?_limit=${limit}&_offset=${offset}`,
      providesTags: ["Post"],
    }),
    createPost: builder.mutation<Post, Omit<Post, "id">>({
      query: (post) => ({ url: "/posts", method: "POST", body: post }),
      invalidatesTags: ["Post"],
    }),
  }),
});
```

USEFUL DESIGN PATTERNS

Conditional Queries

Skip queries when required arguments are missing:

```
useGetPostQuery(postId, {
  skip: !postId,
});
```

Manual Refetching

Execute manual updates on demand:

```
const { data, refetch } =
  useGetPostsQuery({ limit: 5, offset: 0
});

<button onClick={() => refetch()}>
  Refresh
</button>
```

RTK QUERY VS. `CREATEASYNCThunk`

- **Regular Redux Slice:** Client-only UI state (modals, counters).
- **`createAsyncThunk`:** Custom asynchronous workflows that are not cache-centric.
- **RTK Query:** Normalized API server data management, automatic caching, and request tracking.